

Assignment: Advanced Blog Application

Module 10 - Django Model Relationships & Advanced ORM

1. Assignment Overview

Continue working on your Module 9 Blog Application and enhance it with advanced features.

In Module 9, you implemented user authentication, blog post CRUD operations, ownership restrictions, forms, templates, and Django ORM.

In Module 10, you will extend the same project by implementing:

- Model Relationships
- Comments and Nested Replies
- Like/Unlike System
- Post Rating System
- Aggregations and Annotations
- Advanced Queries
- Related Objects and Lookups
- `select_related()`
- `prefetch_related()`
- Query Optimization

Do not create a new project. You must continue and enhance your Module 9 project.

2. Required Features

A. Comment System

Authenticated users should be able to comment on a blog post.

Each comment should contain at least:

Comment

Post

Author

Content

Created At

Updated At

Users should be able to:

- Add comments
- View comments
- Edit their own comments
- Delete their own comments

A user must not be able to edit or delete another user's comment.

B. Comment Reply System

Users should be able to reply to comments.

The system should support nested replies.

Example:

Comment

└─ Reply

└─ Reply

└─ Reply

Example:

Rahim:

How can I learn Django?

Karim:

Start with Django Models.

Rahim:

Should I learn Models before Forms?

Karim:

Yes, that is recommended.

Use an appropriate model relationship to implement this feature.

3. Like / Unlike System

Implement a Like/Unlike system for blog posts.

Authenticated users should be able to:

- Like a post
- Unlike a post
- See the total number of likes

A user must not be able to create multiple likes for the same post.

Example:

How to Learn Django

Likes: 25

[Like]

After liking:

Likes: 26

[Unlike]

4. Comment Like System

Users should also be able to like comments.

Example:

This tutorial is very helpful.

Likes: 12

[Like]

A user should only be able to like a particular comment once.

5. Post Rating System

Implement a 1–5 star rating system for blog posts.

Authenticated users should be able to:

- Give a rating from 1 to 5
- View the average rating
- Update their existing rating

Example:

How to Learn Django

★★★★★ 4.6 / 5

Total Ratings: 125

A user should not create multiple ratings for the same post.

6. Aggregation and Annotation

Use Django ORM Aggregation and Annotation to display useful statistics.

Your project should display information such as:

- Total comments
- Total likes
- Total ratings
- Average rating
- Number of posts written by a user

Example:

Post Title: Django ORM Tutorial

Comments: 25

Likes: 75

Ratings: 40

Average Rating: 4.7

You must use Django ORM functions such as:

Count()

Avg()

Sum()

Min()

Max()

where appropriate.

7. Popular Posts

Create a Popular Posts section.

The page should display posts based on engagement such as:

- Number of likes
- Number of comments
- Average rating

Example:

Popular Posts

1. Django Authentication

2. Django ORM

3. Django Model Relationships

4. Django Forms

Use Annotation and Ordering instead of manually calculating the results in Python.

8. Search and Filtering

Improve the existing blog application by adding post search.

Users should be able to search by:

- Post title
- Post content
- Author

Also implement filtering by category if your project contains categories.

9. Advanced Django ORM Queries

Your project must demonstrate advanced queries using Django ORM.

Implement examples such as:

Posts written by a specific user

`author__username`

Posts containing a specific keyword

`title__icontains`

`content__icontains`

Posts with a minimum number of likes

`likes__...`

Posts with ratings greater than or equal to 4

`ratings__rating__gte=4`

Posts with more than a specific number of comments

Use:

`annotate()`

`Count()`

`filter()`

Students should demonstrate appropriate field lookups and related-object lookups.

10. Related Objects and Lookups

Use Django ORM relationships to retrieve related objects.

Your project should demonstrate queries involving:

User → BlogPost

BlogPost → Comment

Comment → Reply

BlogPost → Like

BlogPost → Rating

Comment → CommentLike

Use appropriate related lookups such as:

`author__username`

`comments__author`

`comments__content`

`Ratings__rating`

11. Query Optimization

Optimize your database queries using:

`select_related()`

Use `select_related()` where appropriate for relationships such as:

BlogPost → Author

`prefetch_related()`

Use `prefetch_related()` where appropriate for relationships such as:

BlogPost → Comments

BlogPost → Likes

BlogPost → Ratings

The goal is to reduce unnecessary database queries.

Students should be able to explain why they used `select_related()` or `prefetch_related()`.

12. User Profile Statistics

Enhance the existing user/profile functionality.

The profile page should display:

Username: Rahim

Posts: 15

Comments: 48

Likes Given: 125

Ratings Given: 20

Use Django ORM aggregation where appropriate.

13. Django Admin

Register the newly created models in Django Admin.

At minimum, the following should be manageable from Admin:

BlogPost

Comment

PostLike

CommentLike

PostRating

Category (if implemented)

Customize the Admin interface to display useful information such as:

Post Title

Author

Created Date

Comment Count

Like Count

Rating

14. Suggested Pages

Your Module 10 project should contain at least:

1. Home Page

2. Post Details Page
3. Create Post Page
4. Edit Post Page
5. Delete Post Page
6. My Posts Page
7. User Profile Page
8. Search Results Page

You may create additional pages when necessary.

15. Technical Requirements

The project must use:

- Python
- Django
- Django Templates
- Django Forms
- Django ORM
- SQLite
- Django Authentication System
- HTML/CSS
- Bootstrap or Custom CSS

Do not use Django REST Framework. This follows the Module 9 project requirement.

16. Important Rules

1. Continue your Module 9 project.
2. Do not create a completely separate project.
3. All existing Module 9 features must continue working.
4. Proper model relationships must be used.
5. Do not store comments, likes, or ratings as plain text fields inside BlogPost.
6. Implement proper authorization for editing/deleting user-owned content.
7. Prevent duplicate likes.
8. Prevent duplicate ratings from the same user for the same post.
9. Use Django ORM instead of writing unnecessary Python loops.
10. Use `select_related()` and `prefetch_related()` where appropriate.
11. Keep your project structure clean.

17. Submission Guidelines

Submit the same GitHub repository updated with Module 10 features.

Your repository should include the complete source code, migrations, requirements.txt, README, and proper project structure, consistent with the Module 9 submission requirements.

Optional

You may also submit:

- Project demonstration video
- UI screenshots
- Additional documentation